FDS Pokémon Battles Prediction 2026

Azamat Adenov
Sapienza University of Rome, Rome, Italy
azamatadenov9@gmail.com

Nurkhan Saret
Sapienza University of Rome, Rome, Italy
nuriks139@gmail.com

Diana Sarsembayeva
Sapienza University of Rome, Rome, Italy
sarsembayeva.2238069@studenti.uniroma1.it

Ameliya Sultanova
Sapienza University of Rome, Rome, Italy
sultanova.2303272@studenti.uniroma1.it

Abstract

Binary prediction of Player 1's victory was studied using 10,000 labelled Pokémon battle logs. The team transformed JSONL records into numerical team-, matchup-, status-, move- and timeline-based features, then evaluated linear, tree-based and ensemble classifiers with stratified five-fold validation. The final notebook generated its submission with a Logistic Regression–LightGBM–HistGradientBoosting stack and a threshold of 0.515 (OOF accuracy 0.6792). The team's best Kaggle result was 0.68533 (rank 45); the supplied leaderboard screenshot does not identify the model behind that score.

1. Introduction

The FDS Pokémon Battles Prediction 2026 task is to classify `player_won` from JSONL battle logs (10,000 training and 5,000 test records). Each record combines team information with a turn-by-turn battle timeline. The project objective was to represent relative team strength and battle progression as features suitable for supervised classification.

2. Evaluated Approaches

2.1 Feature Engineering & Selection

Initial extraction produced 42 candidate predictors (44 columns including the identifier and target), covering team base statistics, estimated team HP, momentum, revealed and unrevealed Pokémon, fainting, status, switches, boosts, hazards and move power. Mann–Whitney U tests for continuous features and chi-square tests for low-cardinality features [3] retained 29 predictors at $\alpha = 0.05$. Correlation pruning reduced these to 21; RFECV [1] selected 30 of 32 features after adding HP-ratio and momentum interactions. Neither refinement improved the corresponding voting benchmark (0.6742 and 0.6732 versus 0.6754 initially). A separate 48-feature asymmetric timeline representation summarized the last recorded states, HP changes, status, switches and tactical actions. Figure 1 shows selected correlations from the earlier feature-selection analysis, not importance scores from the final model.

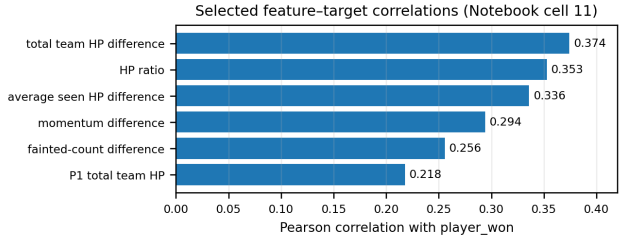


Figure 1: Selected Pearson correlations with `player_won` from the earlier feature-selection analysis. These are associations, not final-model feature importances.

2.2 Model Experimentation & Refinement

On the initial feature set, five-fold accuracy was 0.6772 for Logistic Regression, 0.6704 for LightGBM, 0.6625 for Random Forest and 0.6754 for soft voting. On the RFECV-derived feature set, OOF blend threshold tuning increased accuracy from 0.6747 (threshold 0.50) to 0.6772 (threshold 0.466). A 20-configuration LightGBM [2] randomized search reported ROC-AUC 0.7173; this is a different metric from accuracy. On the 48-feature representation, Logistic Regression scored 0.6799, Random Forest 0.6626, HistGradientBoosting 0.6729 and LightGBM 0.6726. A probability blend reached 0.6788 at threshold 0.478. The subsequent StackingClassifier combined scaled Logistic Regression, regularized LightGBM and HistGradientBoosting, with a Logistic Regression meta-learner and passthrough features; at threshold 0.515 it scored 0.6792. Logistic Regression therefore had the highest recorded accuracy in this comparison, although the notebook used stacking to generate the final submission. The notebook does not document a further empirical reason for preferring stacking.

A separate 13-feature domain branch reached 0.6608 with tuned voting. A 60-feature branch adding exponentially time-weighted HP summaries, late-faint counts and CountVectorizer/TruncatedSVD [1] team-composition features reached 0.6755. A 46-feature classic branch included hand-crafted battle mechanics and three smoothed roster-level target-encoding aggregates; Logistic Regression reached 0.6796 (ROC-AUC 0.7486), while its tuned ensemble reached 0.6790. These separate pipelines are not directly comparable, and the classic branch's encoding procedure has validation-leakage concerns described below.

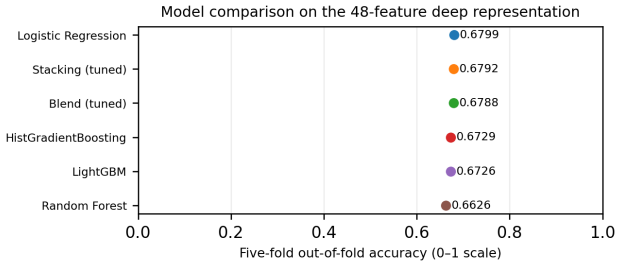


Figure 2: Five-fold OOF accuracy on the 48-feature timeline representation. Logistic Regression recorded 0.6799; the stacking pipeline used for submission recorded 0.6792. The horizontal axis uses the full 0–1 scale.

2.3 Validation Strategy

Model evaluation used StratifiedKFold with five shuffled folds (`random_state=42`); `cross_val_predict` provided OOF probabilities for the blend and stack. Accuracy was the main competition metric, with ROC-AUC reported in selected experiments. Some supervised feature screening and RFECV selection were performed before later CV evaluations, so validation folds influenced feature selection. In the classic branch, roster encodings were constructed using internal folds, but the full-data target mean was used as the smoothing prior and the encoded matrix was then reused in a separate CV evaluation; its scores may therefore be optimistic. The RFECV-branch blend and stacking thresholds were each optimized on the same OOF predictions used to report their accuracies; these are threshold-tuned estimates rather than independent generalization estimates.

McNemar's test [4] returned $p = 0.911$, but the notebook compared a screened-feature Logistic Regression baseline with earlier blend probabilities thresholded using the stack's threshold; it did not test the final stack against that baseline. Finally, features summarize completed battle timelines, so the task is retrospective outcome classification rather than pre-battle or in-battle forecasting.

3. Results

The final notebook pipeline fitted the stacking ensemble on all 10,000 training battles and applied threshold 0.515 to the 5,000 test battles to create `submission.csv`. Its recorded OOF accuracy was 0.6792; the highest OOF accuracy in the 48-feature comparison was 0.6799 for Logistic Regression. The Kaggle leaderboard screenshot shows a team-best score of 0.68533 at rank 45 and a latest score of 0.67600. Because the screenshot does not link a score to a particular submission file, the best leaderboard score is reported as a team result, not attributed specifically to stacking.

References

[1] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[2] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," *Advances in Neural Information Processing Systems 30*, 2017.

[3] P. Virtanen et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[4] S. Seabold and J. Perktold, "Statsmodels: Econometric and Statistical Modeling with Python," *Proceedings of the 9th Python in Science Conference*, 2010.